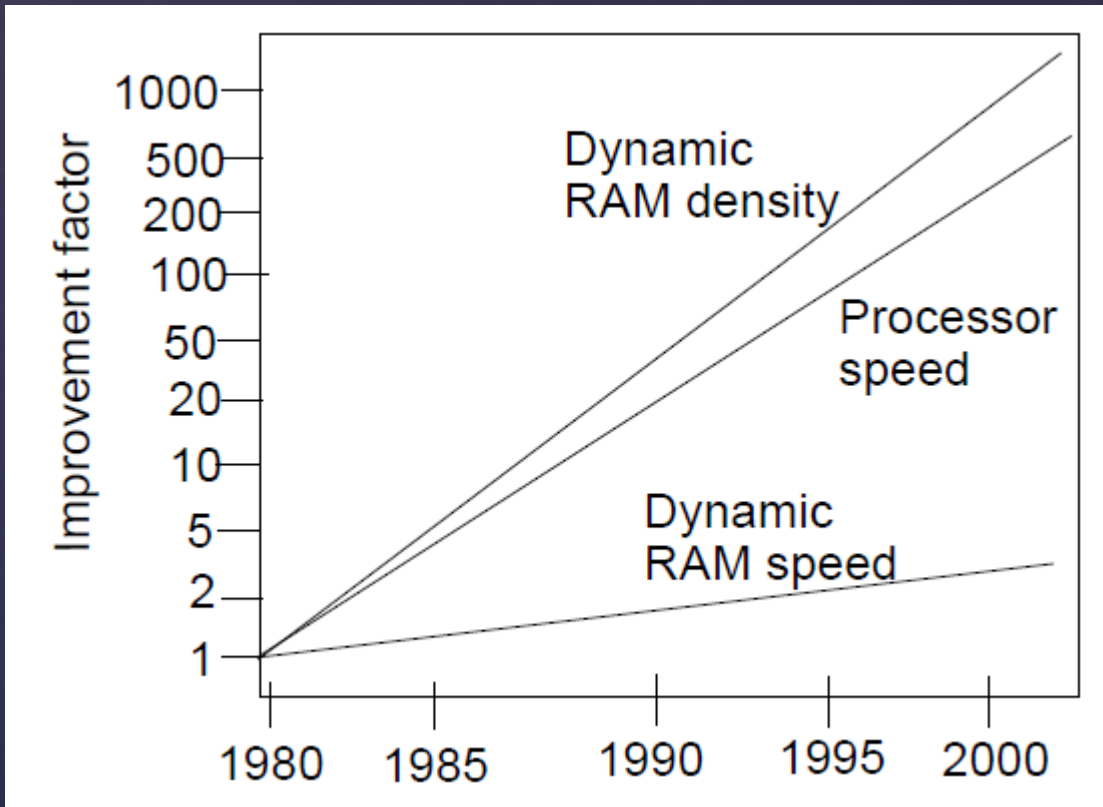


Moore's Law

Gordon Moore famously said “The number of transistors on a single integrated circuit chip doubles every 12-18 months”.

People **think** that means “computer power doubles every 12-18 months”. It doesn't.

Moore's Law



This plot shows the basic trend of performance over time

Notice how **RAM speed** lags behind **RAM size** and **processor speed**

Conclusion? RAM IS BAD.

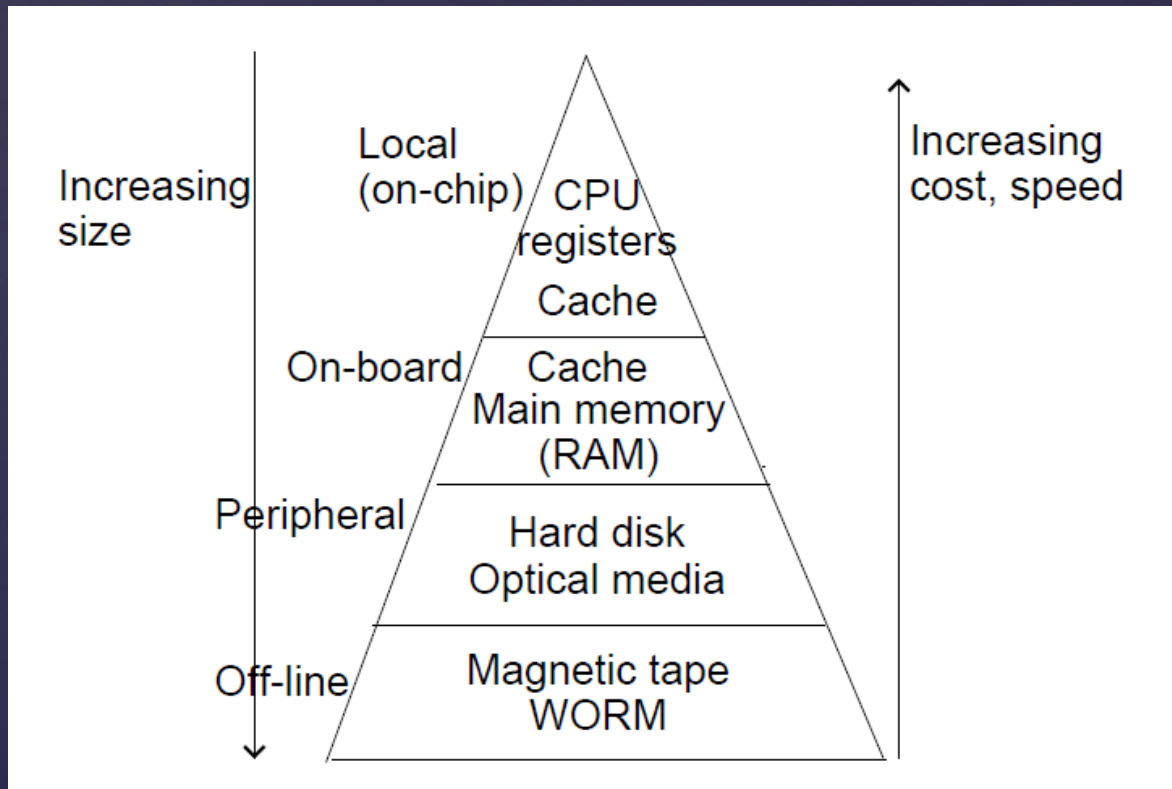
Memory

In principle, **memory** means any place in a computer that stores information (data and/or instructions).

The fundamental characteristics that define memory are size, speed and cost. Ideally we would have a machine with unlimited storage, and negligible access times for free!

It is a sad truth that we can't always get what we want. Cost (per word) is a function of speed. Fast memory is expensive and slow memory is cheap.

Memory Hierarchy



Design challenge:
Can we design a system that **acts** like it has that **infinitely large, infinitely fast** memory?

The Principle of Locality

As usual, the answer is “kind of”. It is a principle of computer science that data and code accesses tend to be:

Local in time: Words that have been accessed recently are likely to be accessed again

Local in space: Words near words that have been accessed recently are likely to be accessed soon

This is not a law of the universe. It results because of the way we organize software (modules, methods, objects, loops, structures, arrays and so on).

However, we can **exploit** the principle of locality to enhance performance.

Types of memory

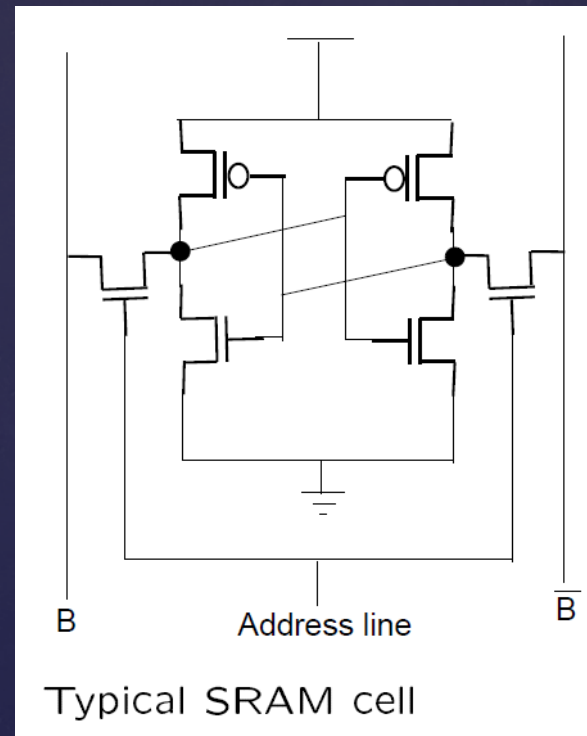
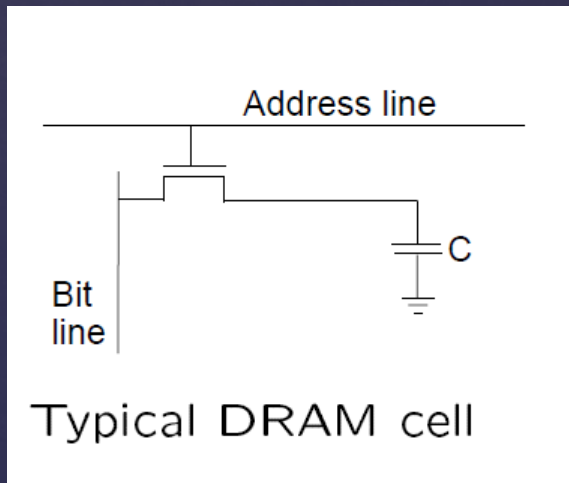
There are basically two kinds of memory in the world. In both cases, their common names are lies, even though they are generally distinguishable.

RAM (Random Access Memory): is **volatile** (it loses its value when powered off) and is **easy** to both **read and write**

ROM (Read Only Memory): is **non-volatile** and is **easy to read** but **difficult to write**

Types of RAM

ROM comes in many varieties that we will look at later, but RAM basically has only two variants: Dynamic RAM (DRAM) and Static RAM (SRAM)



DRAM vs SRAM

DRAM is analog and tends to be inexpensive and extremely dense. Most modern computers use some form of DRAM as their main memory.

SRAM is digital and tends to be much more expensive, but faster. Cache memories are usually made of SRAM.

Locality: An example

Suppose a computer has two levels of memory. L1 is 1K words in size and has an access time of $0.01\mu\text{s}$. L2 is 100K words in size and has an access time of $0.1\mu\text{s}$. Assume:

1. If the word we need is **in L1**, read it.
2. If the word we need is **NOT in L1**, read it from L2 to L1, **then** from L1

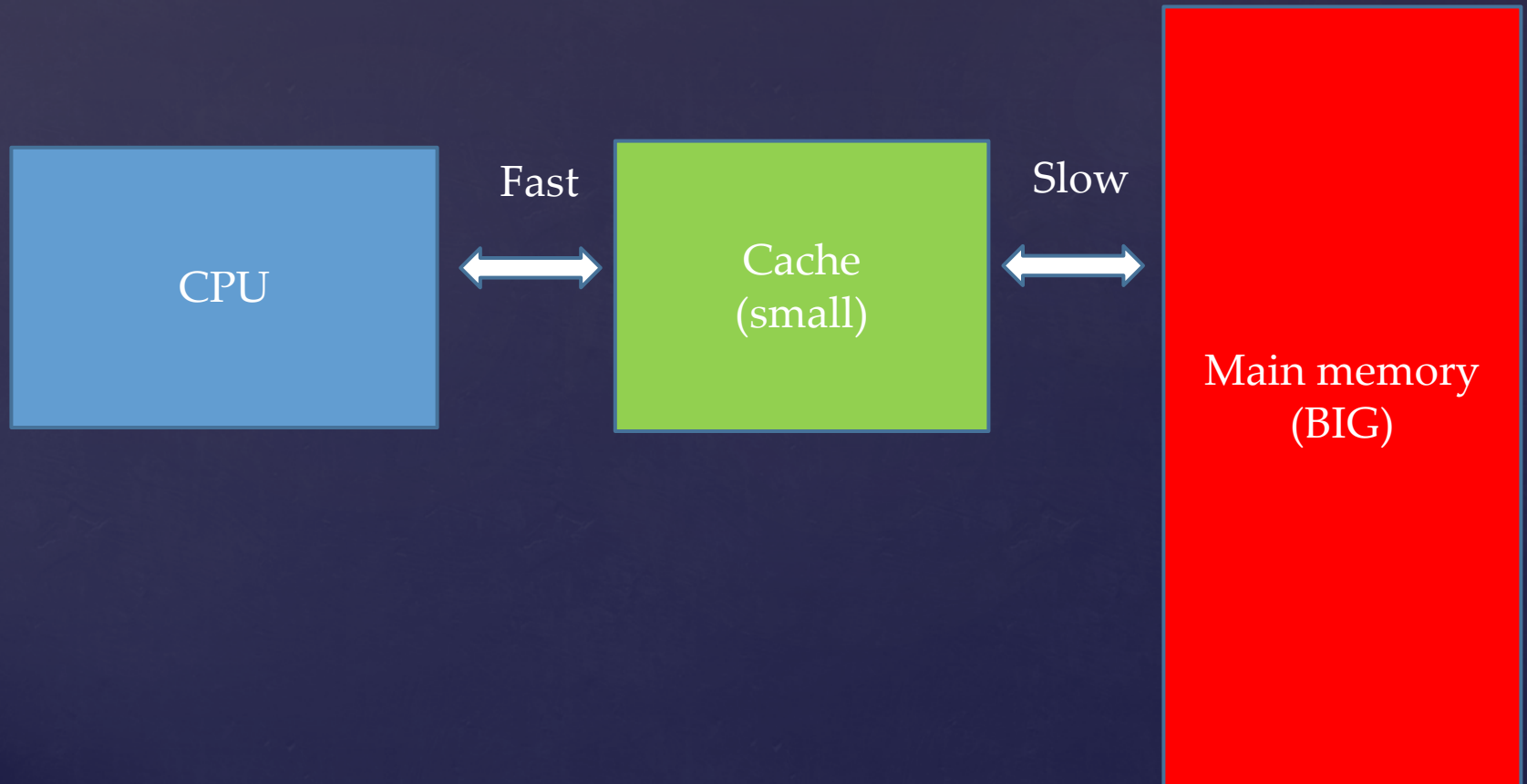
With probability **H**, we will find the data in L1.

Questions:

1. What is the average access time
2. For what value of H does the overall access time become less than the access time of L2 alone?
3. Suppose L1 memory is 10 times more expensive than L2. What is the cost effect?
4. What value of H makes this pay off?

The cache

This example illustrates the basic idea behind **cache** systems.



Cache Lines

Caches have a block structure.

Line Number	Tag	Content
0		Content depends on organization. Each line will have k words
1		
2		
3		
4		
...		
C-1		

Locality: If we need a **word**, we copy the entire **line** where that line is located.

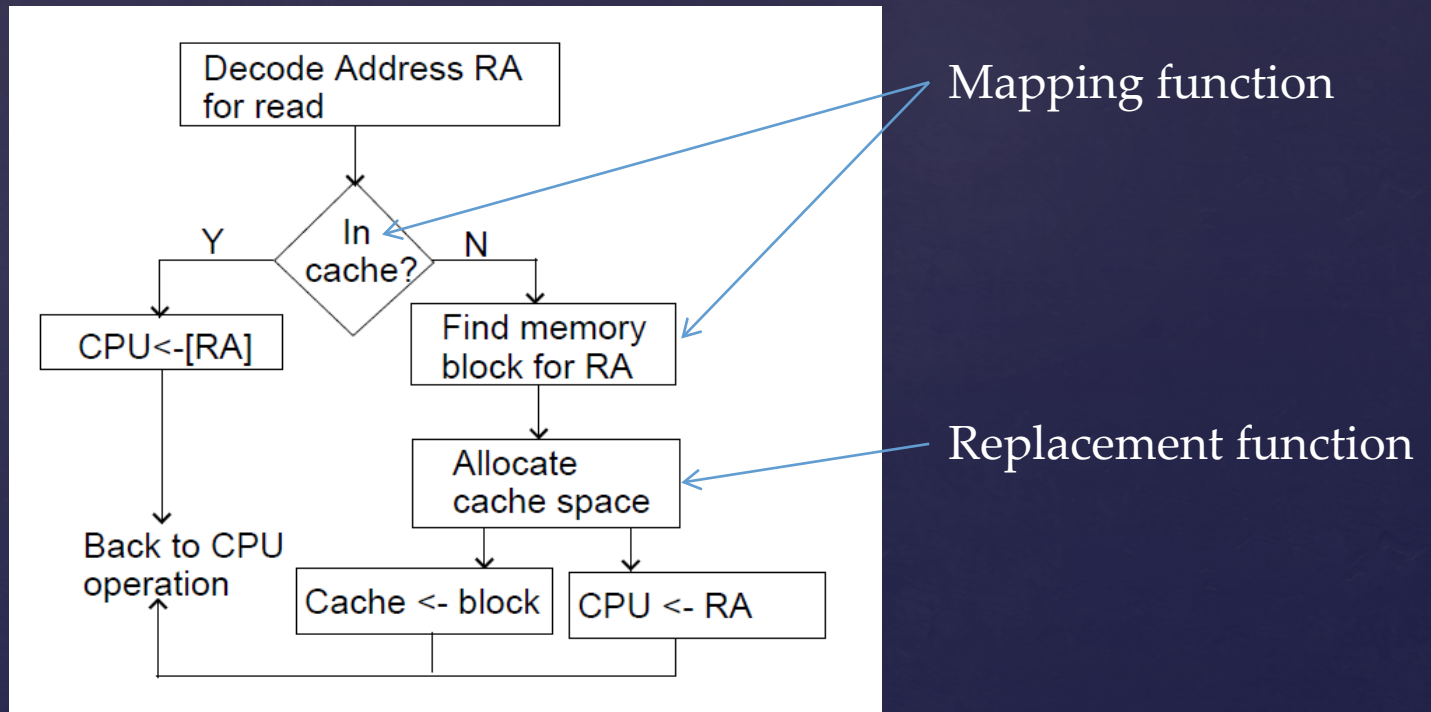
We will say caches have **C lines** of **k words** each.

For **N address lines**, main memory will have $2^N/k \gg C$ lines (blocks)

This all happens **entirely** in hardware and is **completely invisible** to the software.

Cache Operation (Read)

Caches have a block structure.



This all happens **entirely** in hardware and is **completely invisible** to the software.

Cache Design Decisions

When designing a cache system, we will have decisions to make:

Mapping function: Where & how do we locate words in cache?

Replacement algorithm: What do we do if the cache is full?

Write policy: How do we keep cache & main memory synchronized?

Line size: How many words on a line?

Organization: How many caches? How many levels?

Mapping Functions

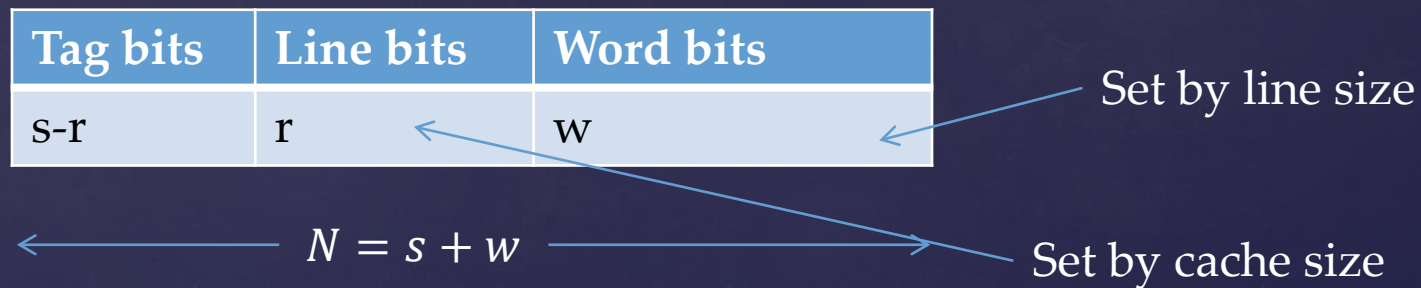
We basically have three options:

In **Direct Mapping**, we map each block of memory to **one and only one** line of cache. In math, we say:

$$i = j \bmod C$$

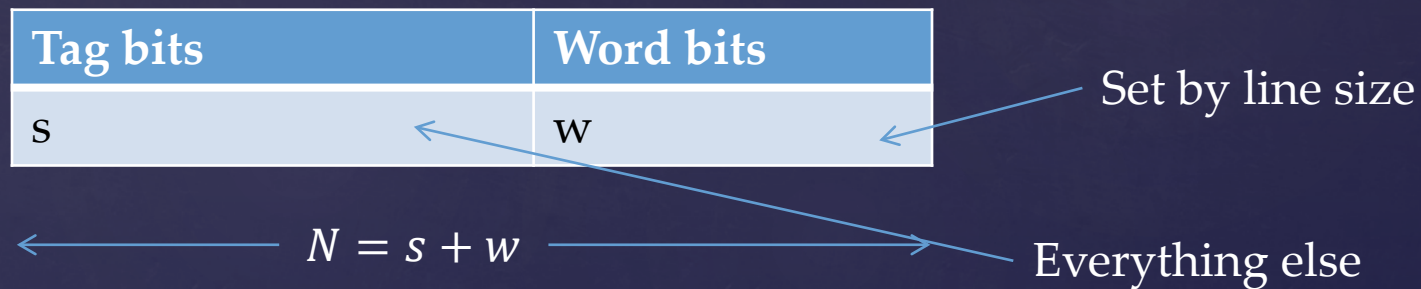
Where i is the line number in cache, j is the line number in main memory and C is the number of lines in the cache.

As usual, the math is more complicated than what is actually going on. Basically we are subdividing the address as follows:



The w lowest order bits are used to index **into the line**.
The remaining s bits are divided into r bits to **select the line**,
and $s-r$ bits to **identify** it.

Associative mapping is the other extreme from direct mapping. In associative mapping, we can map **any** memory block to **any** cache line. All the non word bits are tag.



The tag bits remember the rest of the address so we know where it came from

Set-associative mapping is somewhere in the middle. In set-associative mapping, we associate a group (set) of lines with each memory block. The block must map to the set, but can map to any line within the set. In math:

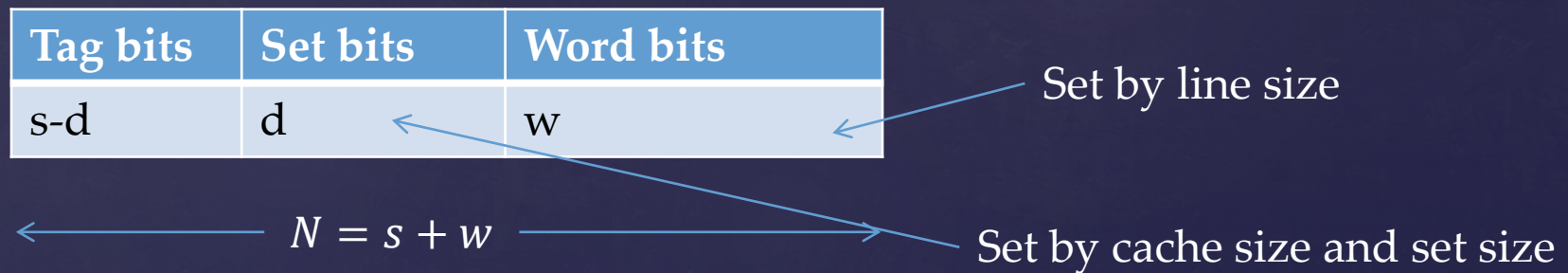
$$C = v \times K$$

Cache is v sets of K lines each

$$i = j \bmod v$$

The j block always maps to the i set, but we have flexibility within the set

As usual, the math is more complicated than what is actually going on. Basically we are subdividing the address as follows:



The w lowest order bits are used to index **into the line**.
The remaining s bits are divided into d bits to **select the set**, and $s-d$ bits to **identify** it.

Mapping Functions Discussion

Direct mapping: Simplest. Most likely to collide.

Associative: Most complicated. Needs a replacement algorithm

Set-associative: Best of both/worst of both

Cache example

The CPU can access a 64K word memory space. It uses a 128 word cache, organized into lines of 16 words per line. The following operations are performed:

Operation	Address
Read	\$4425
Read	\$4415
Read	\$0429
Read	\$442A
Read	\$042F
Read	\$182E

Analyze the problem assuming:

- 1) Direct mapping
- 2) Associative mapping
- 3) 2-way set associative mapping

Replacement algorithms

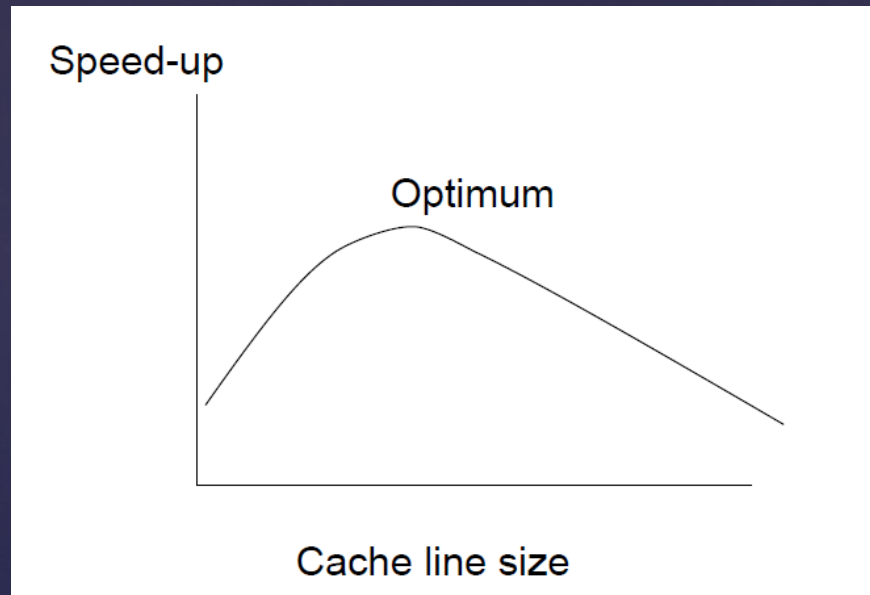
Eventually, we are going to have a cache **collision**. That means there is already data on the line where we need to copy the new data. What do we do?

Write Policy

Eventually the CPU is going to **write** to memory. Presumably, to save time, we write **into the cache**. What happens to main memory? What happens when we have a collision on that line?

Line size

As with so many things, we have a trade-off. Speed-up with line size looks like this:



Let's talk about why...

Cache organization

Cache systems can be further subdivided:

- Separate **I-cache** and **D-cache**. As we will see later, this avoids a bottleneck caused by the need to Fetch each instruction
- L1, L2, L3... Modern computers have multiple levels of cache. As you go up, size increases and speed decreases.

We will re-visit some of these questions later when we talk about **pipelining** and **multi-core** systems